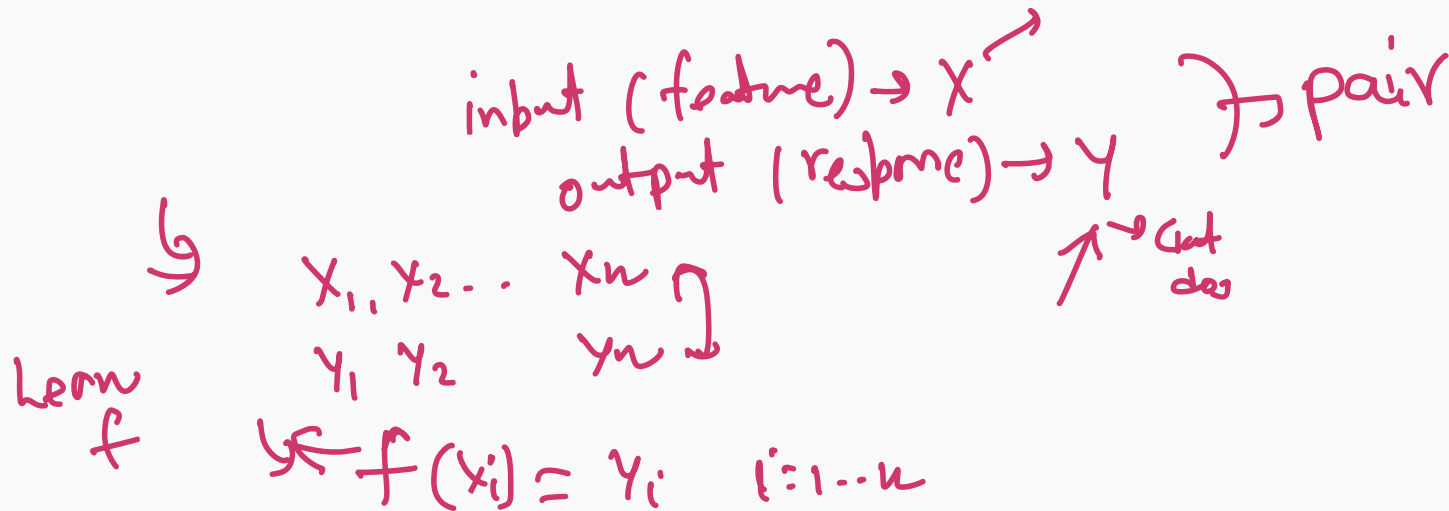


Supervised Learning Fundamentals



Machine Learning Problem Types

Most supervised learning problems fall into two main categories:

Problem type	Output type
✓ Regression	<u>continuous</u> numerical value → y : House price prediction
✓ Classification	<u>discrete</u> category / label → y : $\begin{pmatrix} 1 \rightarrow \text{cat} \\ 0 \rightarrow \text{not} \end{pmatrix}$

⇒ $\underline{x} \rightarrow y$

⇒ $y = f(\underline{x}) \rightarrow \text{anything (numeric or category)}$

numeric Regression: ↙
category classification: ↘

Machine Learning Problem Types

Most supervised learning problems fall into two main categories:

Problem type	Output type
Regression	continuous numerical value
Classification	discrete category / label

Key idea: Choose based on the target variable y .

What is Regression?

Regression models predict continuous numerical values.

What is Regression?

Regression models predict **continuous** numerical values.

Examples:

- House **price** prediction

What is Regression?

Regression models predict **continuous** numerical values.

Examples:

- House **price** prediction
- Student **marks/GPA** forecasting

What is Regression?

Regression models predict **continuous** numerical values.

Examples:

- House **price** prediction
- Student **marks/GPA** forecasting
- **temperature** or **stock price** prediction

What is Regression?

Regression models predict **continuous** numerical values.

Examples:

- House **price** prediction
- Student **marks/GPA** forecasting
- **temperature** or **stock price** prediction
- **sales revenue** estimation

What is Regression?

Regression models predict **continuous** numerical values.

Examples:

- House **price** prediction
- Student **marks/GPA** forecasting
- **temperature** or **stock price** prediction
- **sales revenue** estimation

What is Regression?

Regression models predict **continuous** numerical values.

Examples:

- House **price** prediction
- Student **marks/GPA** forecasting
- **temperature** or **stock price** prediction
- **sales revenue** estimation

Example equation:

$$\text{Marks} = f(\overset{x_1}{\text{Hours Studied}}, \overset{x_2}{\text{Attendance}})$$

Y x₁ x₂

God: unknown & want to train & learn

What is Regression?

Regression models predict **continuous** numerical values.

Examples:

- House **price** prediction
- Student **marks/GPA** forecasting
- **temperature** or **stock price** prediction
- **sales revenue** estimation

Example equation:

$$\text{Marks} = f(\text{Hours Studied}, \text{Attendance})$$

Output: any real number (e.g., 72.5).

What is Classification?

Classification models predict discrete categories (labels).

What is Classification?

Classification models predict **discrete** categories (labels).

Examples:

- Email: Spam vs Not Spam

What is Classification?

Classification models predict **discrete** categories (labels).

Examples:

- Email: **Spam** vs Not Spam
- Transactions: **Fraud** vs Genuine
 1 0

What is Classification?

Classification models predict **discrete** categories (labels).

Examples:

- Email: **Spam** vs Not Spam
- Transactions: **Fraud** vs Genuine
- Medical: **Disease** vs Healthy

What is Classification?

Classification models predict **discrete** categories (labels).

Examples:

- Email: **Spam** vs Not Spam
- Transactions: **Fraud** vs Genuine
- Medical: **Disease** vs Healthy
- Images: **Cat** vs Dog

What is Classification?

Classification models predict **discrete** categories (labels).

Examples:

- Email: **Spam** vs Not Spam
- Transactions: **Fraud** vs Genuine
- Medical: **Disease** vs Healthy
- Images: **Cat** vs Dog

What is Classification?

Classification models predict **discrete** categories (labels).

Examples:

- Email: **Spam** vs Not Spam
- Transactions: **Fraud** vs Genuine
- Medical: **Disease** vs Healthy
- Images: Cat vs Dog, Horse, any other animal

Output: one class from a predefined set (e.g., {spam, not spam}).

Regression vs Classification

Aspect	Regression	Classification
Output	<u>continuous</u> (e.g., 87.4)	<u>categorical</u> (e.g., Pass/Fail)
Examples	<u>Price, Marks, Temperature</u>	<u>Spam, Disease, Customer churn</u>
<u>Metrics</u>	<u>MAE</u> , <u>RMSE</u> , <u>R^2</u> ✓	<u>Accuracy</u> , <u>Precision</u> , <u>Recall</u> , <u>F1</u> , <u>AUC</u>

Regression vs Classification

Aspect	Regression	Classification
Output	continuous (e.g., 87.4)	categorical (e.g., Pass/Fail)
Examples	Price, Marks, Temperature	Spam, Disease, Customer churn
Metrics	MAE, RMSE, R^2	Accuracy, Precision, Recall, F1, AUC

Tip: Identify the problem type early — it determines model choice and evaluation.

A Typical Machine Learning Dataset

Example: Student performance data

	x_1	x_2	y
	Hours	Attendance	Marks (target)
$x_{11} \rightarrow$	2	x_{21} 70 $\rightarrow$	$\rightarrow$ 55 $\rightarrow y_1$
$\rightarrow 4$ x_{12}		$\rightarrow$ 80 x_{22}	$\rightarrow$ 65 $- y_2$
6 x_{13}		85 x_{23}	78 y_3
8 x_{14}		95 y_{24}	90 y_4

feature matrix: $X = \begin{bmatrix} x_{11} & x_{21} \\ x_{12} & x_{22} \\ x_{13} & x_{23} \\ x_{14} & x_{24} \end{bmatrix}$, $y = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix}$

x_1 x_2

A Typical Machine Learning Dataset

Example: Student performance data

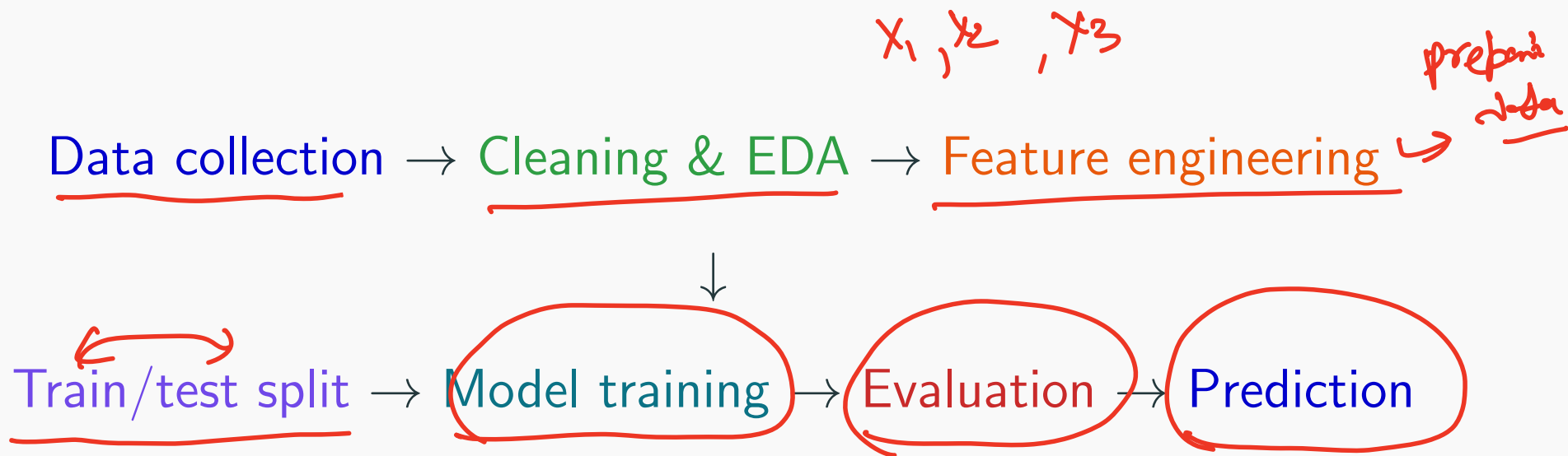
Hours	Attendance	Marks (target)
2	70	55
4	80	65
6	85	78
8	95	90

$y = f(x)$
→ 3 → train
n = 1 → testing

Features X : x_1 x_2
Hours, Attendance

Target y : Marks

The Machine Learning Pipeline



This pipeline applies to regression, classification, and deep learning.

Linear Regression Intuition

The simplest and most interpretable **regression** model:

$\hat{w}_1, \dots, \hat{w}_p$: estimated / known

$y = f(x)$

$\hat{y} = \hat{w}_1 x_1 + \hat{w}_2 x_2 + \dots + \hat{w}_p x_p + b$

$$\hat{y} = w_1 x_1 + w_2 x_2 + \dots + b$$

$x_1, \dots, x_n \rightarrow$ feature
 $y_1, \dots, y_n \rightarrow$ output

LR $\rightarrow f() = w_1 x_1 + w_2 x_2 + \dots + b$

$y_i = f(x_i)$

$w_1 x_1 + w_2 x_2 + \dots + b$

$w_1 x_1$ $w_2 x_2$ $w_p x_p$

w_1 w_2 w_p

x_1 x_2 x_p

$\hookrightarrow f() = f_1 \circ f_2 \circ \dots \circ f_k()$ $\rightarrow$ NNs

w_1 w_2 w_k

b_1 b_2 b_k

unknown weights

weights and biases in NNs

Linear Regression Intuition

The simplest and most interpretable **regression** model:

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + b$$

Meaning of terms:

- **w**: **weights** (importance of each feature)

$$\Rightarrow \left[\begin{array}{l} y_1 = w_1x_{11} + w_2x_{21} + b \\ y_2 = w_1x_{12} + w_2x_{22} + b \\ y_3 = w_1x_{13} + w_2x_{23} + b \\ y_4 = w_1x_{14} + w_2x_{24} + b \end{array} \right]$$

Linear Regression Intuition

The simplest and most interpretable **regression** model:

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + b$$

Meaning of terms:

- **w**: **weights** (importance of each feature)
- **b**: **bias** (offset)

Linear Regression Intuition

The simplest and most interpretable **regression** model:

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + b$$

Meaning of terms:

- **w**: **weights** (importance of each feature)
- **b**: **bias** (offset)
- **$\hat{y}$** : **prediction**

Linear Regression Intuition

The simplest and most interpretable **regression** model:

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + b$$

Meaning of terms:

- **w**: **weights** (importance of each feature)
- **b**: **bias** (offset)
- **$\hat{y}$** : **prediction**

Linear Regression Intuition

The simplest and most interpretable **regression** model:

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + b$$


Meaning of terms:

- **w**: **weights** (importance of each feature)
- **b**: **bias** (offset)
- $\hat{y}$: **prediction**

This should look familiar from the NumPy linear model example.

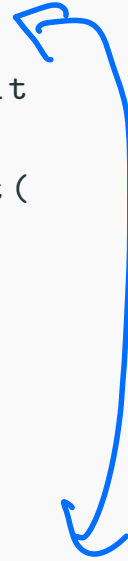
Minimal Scikit-Learn Example (Regression)

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

predictions = model.predict(X_test)
```



Minimal Scikit-Learn Example (Regression)

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

predictions = model.predict(X_test)
```

Output (example):

Model trained successfully.

Predictions: [62.3 78.1 91.5 ...]

Key idea: same `fit` → `predict` workflow appears in most ML libraries.

Linear regression (no non-linearity):

$$\hat{y} = wx + b$$

From Regression to Neural Networks

Linear regression (no non-linearity):

$$\hat{y} = wx + b$$

Single neuron (adds **activation**):

$$\hat{y} = \text{Activation}(wx + b)$$

From Regression to Neural Networks

Linear regression (no non-linearity):

$$\hat{y} = wx + b$$

Single neuron (adds **activation**):

$$\hat{y} = \text{Activation}(wx + b)$$

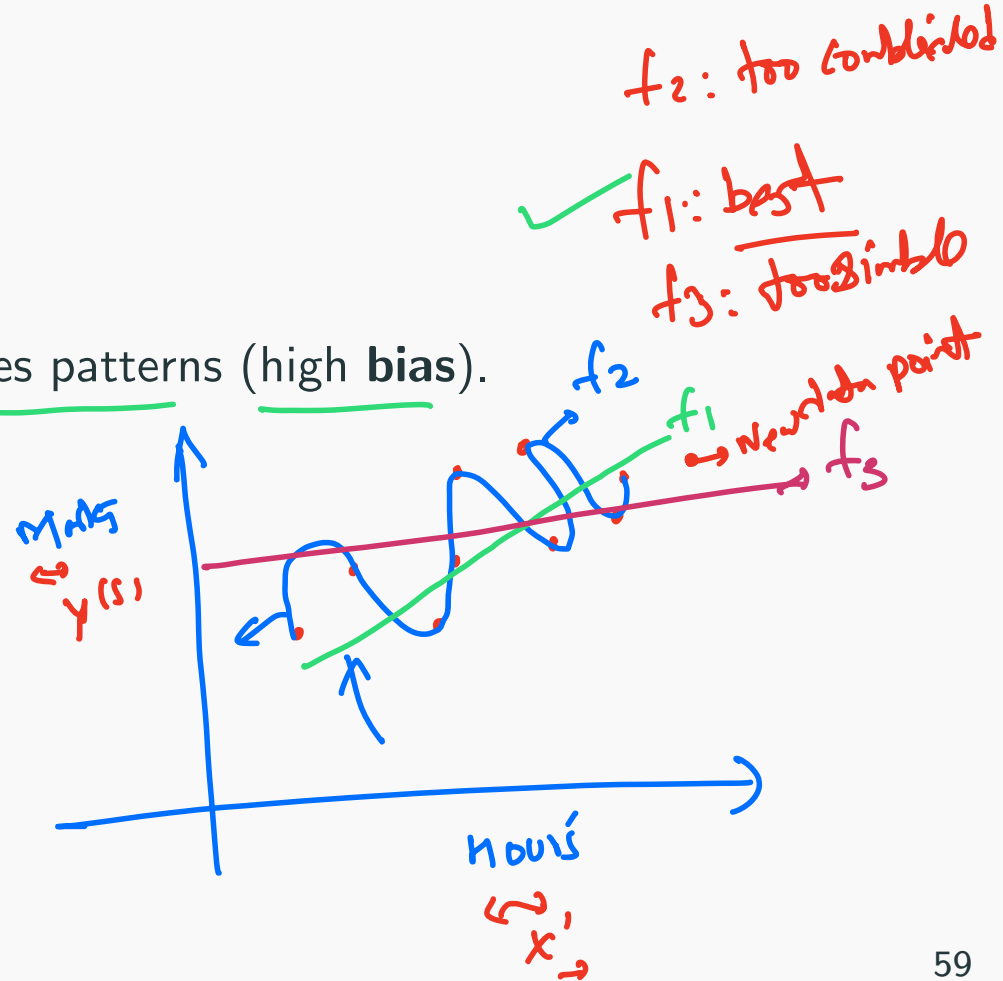
Stacking many neurons + **non-linear activations** $\Rightarrow$ powerful deep learning models.

Overfitting vs Underfitting

→ f_3

- **Underfitting**: model too simple $\Rightarrow$ misses patterns (high **bias**).

$$y = f(x)$$



Overfitting vs Underfitting

- **Underfitting**: model too simple $\Rightarrow$ misses patterns (high **bias**).
- **Overfitting**: model too complex $\Rightarrow$ memorizes training data (high **variance**).

f_2

n data point
 $f \rightarrow (n-1)$ degree of polynomial
 $\nwarrow$
 $n=1000$
 $999 \rightarrow$ parameter

Overfitting vs Underfitting

- **Underfitting**: model too simple $\Rightarrow$ misses patterns (high **bias**).
- **Overfitting**: model too complex $\Rightarrow$ memorizes training data (high **variance**).

Overfitting vs Underfitting

- **Underfitting**: model too simple $\Rightarrow$ misses patterns (high **bias**).
- **Overfitting**: model too complex $\Rightarrow$ memorizes training data (high **variance**).

Goal: generalize well to **unseen** data.

f_1

Train, Validation & Test Sets

Standard split (rule of thumb):

- Training: 70–80%

learn parameters (w, b)

Train, Validation & Test Sets

Standard split (rule of thumb):

- **Training:** 70–80%
- **Validation:** 10–15%

learn parameters (w, b)

tune hyperparameters

Train, Validation & Test Sets

Standard split (rule of thumb):

- **Training:** 70–80%
- **Validation:** 10–15%
- **Test:** 10–20%

$n=100$
75 → train ($w's, b's$)
→ estimate / learn

learn parameters (w, b)

tune hyperparameters

final unbiased evaluation

10 data

15

$x_{86} \dots x_{100}$
 $y_{86}^p = f(x_{86}^p)$
 $y_{100}^p = f(x_{100}^p)$

$y_{86} \dots y_{100} \rightarrow$ ground truth

pred $\rightarrow y_{86}^p, \dots, y_{100}^p$
ground $\rightarrow y_{86}, \dots, y_{100}$

Train, Validation & Test Sets

Standard split (rule of thumb):

- **Training:** 70–80%
- **Validation:** 10–15%
- **Test:** 10–20%

learn parameters (w, b)

tune hyperparameters

final unbiased evaluation

Train, Validation & Test Sets

Standard split (rule of thumb):

- **Training:** 70–80%
learn parameters (w, b)
- **Validation:** 10–15%
tune hyperparameters
- **Test:** 10–20%
final unbiased evaluation

Keeping them separate prevents **data leakage** and gives a reliable estimate of performance.

Key Takeaways — Supervised Learning Fundamentals

- First decide: regression or classification.

Regression & Classification = foundation of supervised learning

Key Takeaways — Supervised Learning Fundamentals

- First decide: **regression** or **classification**.
- Data = features X + target y .

Regression & Classification = foundation of supervised learning

Key Takeaways — Supervised Learning Fundamentals

- First decide: **regression** or **classification**.
- Data = features X + target y .
- **Linear regression** is the starting point for understanding neural networks.

Regression & Classification = foundation of supervised learning

Key Takeaways — Supervised Learning Fundamentals

- First decide: **regression** or **classification**.
- Data = features X + target y .
- **Linear regression** is the starting point for understanding neural networks.
- Always evaluate on unseen data (train/val/test split).

Regression & Classification = foundation of supervised learning

Key Takeaways — Supervised Learning Fundamentals

- First decide: **regression** or **classification**.
- Data = features X + target y .
- **Linear regression** is the starting point for understanding neural networks.
- Always evaluate on **unseen** data (train/val/test split).
- Avoid **overfitting**; aim for **generalization**.

Regression & Classification = foundation of supervised learning

Model Evaluation

Why Model Evaluation Matters

After training a model, the critical question is:

How good is this model on **unseen** data? → f_1
 f_2
 f_3

Why Model Evaluation Matters

After training a model, the critical question is:

How good is this model on **unseen data?**

A good model must:

- Perform well on **unseen** examples (not memorization)

Why Model Evaluation Matters

After training a model, the critical question is:

How good is this model on **unseen data?**

A good model must:

- Perform well on **unseen** examples (not memorization)
- **Generalize** beyond the training set

Why Model Evaluation Matters

After training a model, the critical question is:

How good is this model on **unseen data?**

A good model must:

- Perform well on **unseen** examples (not memorization)
- **Generalize** beyond the training set
- Produce reliable and trustworthy predictions

Why Model Evaluation Matters

After training a model, the critical question is:

How good is this model on **unseen data?**

A good model must:

- Perform well on **unseen** examples (not memorization)
- **Generalize** beyond the training set
- Produce **reliable** and **trustworthy** predictions

Why Model Evaluation Matters

After training a model, the critical question is:

How good is this model on **unseen data?**

A good model must:

- Perform well on **unseen** examples (not memorization)
- **Generalize** beyond the training set
- Produce **reliable** and **trustworthy** predictions

Different tasks need different **metrics**.



Evaluation Depends on the Problem

Task	Recommended metrics
Regression ✓	MAE, RMSE, R^2
Classification (balanced)	Accuracy, F1-score
Classification (imbalanced)	Precision, Recall, F1, ROC-AUC

Evaluation Depends on the Problem

Task	Recommended metrics
Regression	MAE, RMSE, R^2
Classification (balanced)	Accuracy, F1-score
Classification (imbalanced)	Precision, Recall, F1, ROC-AUC

No single best metric: choose based on **real-world cost** of errors.

Regression Evaluation Example

Suppose we have **actual** vs **predicted** values:

ground truth

	Actual	Predicted
y_1	80	$\hat{y}_1$ 78
y_2	70	$\hat{y}_2$ 72
y_3	90	$\hat{y}_3$ 88
y_4	60	$\hat{y}_4$ 63

$$\Rightarrow \text{RMSE} = \sqrt{\frac{1}{4} \sum_{i=1}^4 (y_i - \hat{y}_i)^2}$$
$$\Rightarrow \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$
$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

RMSE $\rightarrow f_1$
RMSE $\rightarrow f_2$
RMSE $\rightarrow f_3$

$\Rightarrow$ choose the one with lowest RMSE

Regression Evaluation Example

Suppose we have **actual** vs **predicted** values:

Actual	Predicted
80	78
70	72
90	88
60	63

Evaluation measures **error**: how close predictions are to actual values.

Regression Metrics in Scikit-Learn

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

y_true = np.array([80, 70, 90, 60])
y_pred = np.array([78, 72, 88, 63])

print("MAE:", mean_absolute_error(y_true, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_true, y_pred)))
print("R2 Score:", r2_score(y_true, y_pred))
```



Output:

MAE: 2.25

RMSE: 2.5

R² Score: 0.96875

Lower MAE/RMSE and higher R^2 indicate better performance.

Classification models predict **categories**, so we summarize results using a **confusion matrix**.

Classification models predict **categories**, so we summarize results using a **confusion matrix**.

Goal: understand **what goes right** and **what goes wrong** (not just one number).

Confusion Matrix

$y_1 = 0$
 $y_2 = 1$
 $y_3 = 1$
 $y_4 = 0$
⋮

		y Pred +	y Pred -
y Actual +	1 ✓	TP ✓	FN →
y Actual -	0 ✓	FP ✓	TN →

Confusion Matrix

	Pred +	Pred -
Actual +	TP	FN
Actual -	FP	TN

- **TP** (true positive): predicted positive and actually positive
- **TN** (true negative): predicted negative and actually negative
- **FP** (false positive): predicted positive but actually negative
- **FN** (false negative): predicted negative but actually positive

Classification Metrics in Scikit-Learn

```
from sklearn.metrics import (  
    accuracy_score, precision_score, recall_score, f1_score  
)  
  
y_true = [1, 0, 1, 1, 0, 1]  
y_pred = [1, 0, 0, 1, 0, 1]  
  
print("Accuracy:", accuracy_score(y_true, y_pred))  
print("Precision:", precision_score(y_true, y_pred))  
print("Recall:", recall_score(y_true, y_pred))  
print("F1 Score:", f1_score(y_true, y_pred))
```



Classification Metrics in Scikit-Learn

```
from sklearn.metrics import (  
    accuracy_score, precision_score, recall_score, f1_score  
)  
  
y_true = [1, 0, 1, 1, 0, 1]  
y_pred = [1, 0, 0, 1, 0, 1]  
  
print("Accuracy:", accuracy_score(y_true, y_pred))  
print("Precision:", precision_score(y_true, y_pred))  
print("Recall:", recall_score(y_true, y_pred))  
print("F1 Score:", f1_score(y_true, y_pred))
```

Output:

Accuracy: 0.8333

Precision: 0.75

Recall: 1.0

F1 Score: 0.8571

Why Accuracy Can Be Misleading

Consider a **fraud detection** dataset:

$\gamma^{(1)} \neq 1$

• 990 **legitimate** transactions

$\gamma^{(1)} \neq 0$

• 10 **fraudulent** transactions

never

Why Accuracy Can Be Misleading

Consider a **fraud detection** dataset:

*imbalanced data
problem*

- 990 **legitimate** transactions
- 10 **fraudulent** transactions

A model that predicts **everything as legitimate** gets **99% accuracy** but detects **zero frauds**.

Why Accuracy Can Be Misleading

Consider a **fraud detection** dataset:

- 990 **legitimate** transactions
- 10 **fraudulent** transactions

A model that predicts **everything as legitimate** gets **99% accuracy** but detects **zero frauds**.

For **imbalanced data**, prefer precision, recall, F1, and ROC-AUC.

- **ROC curve**: plots **TPR** vs **FPR** across thresholds
- **AUC**: single number summarizing ranking quality

ROC-AUC

- **ROC curve**: plots **TPR** vs **FPR** across thresholds
- **AUC**: single number summarizing ranking quality

Interpretation:

AUC	Quality
0.5	Random guessing
0.7–0.8	Acceptable
0.8–0.9	Good
0.9+	Excellent

Key Takeaways — Model Evaluation

- Evaluate on unseen test data.
- Pick metrics based on the task and real-world costs.
- Regression: common metrics include MAE/RMSE.
- Classification: common metrics include F1 and ROC-AUC (for probabilistic models).
- Good evaluation prevents deploying unreliable models.

Key Takeaways — Model Evaluation

- Evaluate on **unseen test data**.
- Pick **metrics** based on the task and real-world costs.
- Regression: common metrics include **MAE/RMSE**.
- Classification: common metrics include **F1** and **ROC-AUC** (for probabilistic models).
- Good evaluation prevents deploying **unreliable** models.

Train → Evaluate → Deploy

TensorFlow

Why TensorFlow?

TensorFlow is a popular open-source framework for machine learning and deep learning.

- Speed with GPU/TPU support

Why TensorFlow?

TensorFlow is a popular open-source framework for **machine learning** and **deep learning**.

- **Speed** with **GPU/TPU** support
- **Keras API**: high-level and beginner-friendly

Why TensorFlow?

TensorFlow is a popular open-source framework for **machine learning** and **deep learning**.

- **Speed** with **GPU/TPU** support
- **Keras API**: high-level and beginner-friendly
- **Automatic differentiation** $\Rightarrow$ **backpropagation**

Why TensorFlow?

TensorFlow is a popular open-source framework for **machine learning** and **deep learning**.

- **Speed** with **GPU/TPU** support
- **Keras API**: high-level and beginner-friendly
- **Automatic differentiation** $\Rightarrow$ backpropagation
- **Production** tools for deployment

Why TensorFlow?

TensorFlow is a popular open-source framework for **machine learning** and **deep learning**.

- **Speed** with **GPU/TPU** support
- **Keras API**: high-level and beginner-friendly
- **Automatic differentiation** $\Rightarrow$ backpropagation
- **Production** tools for deployment

Why TensorFlow?

TensorFlow is a popular open-source framework for **machine learning** and **deep learning**.

- **Speed** with **GPU/TPU** support
- **Keras API**: high-level and beginner-friendly
- **Automatic differentiation** $\Rightarrow$ backpropagation
- **Production** tools for deployment

Learning path: NumPy $\rightarrow$ Pandas $\rightarrow$ TensorFlow $\rightarrow$ Deep learning

From Arrays to Tensors

TensorFlow's core data structure is the **tensor** — a generalization of NumPy arrays.

From Arrays to Tensors

TensorFlow's core data structure is the **tensor** — a generalization of NumPy arrays.

- 0D: scalar (single number)
- 1D: vector
- 2D: matrix →
- nD: tensor

From Arrays to Tensors

TensorFlow's core data structure is the **tensor** — a generalization of NumPy arrays.

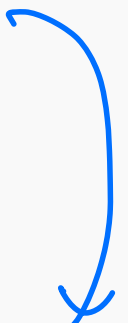
- **0D**: scalar (single number)
- **1D**: vector
- **2D**: matrix
- **nD**: tensor

If you understand **shape** and **axes** in NumPy, you already understand most tensor concepts.



Creating Tensors

```
import tensorflow as tf  
a = tf.constant([1, 2, 3])  
b = tf.constant([4, 5, 6])  
  
print(a)  
print(a + b) \ # elementwise addition
```



Creating Tensors

```
import tensorflow as tf

a = tf.constant([1, 2, 3])
b = tf.constant([4, 5, 6])

print(a)
print(a + b) # elementwise addition ✓
```



Output:

```
tf.Tensor([1 2 3], shape=(3,), dtype=int32)
tf.Tensor([5 7 9], shape=(3,), dtype=int32)
```

Notice: TensorFlow shows **values**, **shape**, and **dtype**.

Tensor Shapes

```
import tensorflow as tf

x = tf.constant([[1, 2, 3],
                 [4, 5, 6]])

print(x.shape)
print(x.dtype)
```

Tensor Shapes

```
import tensorflow as tf

x = tf.constant([[1, 2, 3],
                 [4, 5, 6]])

print(x.shape)
print(x.dtype)
```

Output:

```
(2, 3)
<dtype: 'int32'>
```

Why it matters: neural networks expect specific **input dimensions**.

Integration with NumPy & Pandas

```
import numpy as np
import pandas as pd
import tensorflow as tf

# From NumPy
arr = np.array([1, 2, 3])
t = tf.constant(arr)
~      tf.constant()

# From Pandas DataFrame
# X = df[["feature1", "feature2"]].to_numpy()
# X_tf = tf.constant(X)
```

Integration with NumPy & Pandas

```
import numpy as np
import pandas as pd
import tensorflow as tf

# From NumPy
arr = np.array([1, 2, 3])
t = tf.constant(arr)

# From Pandas DataFrame
# X = df[["feature1", "feature2"]].to_numpy()
# X_tf = tf.constant(X)
```

Idea: Pandas → NumPy → TensorFlow tensors.

Key Takeaways — TensorFlow & Neural Networks

- **Tensors** are the core data structure (like NumPy arrays).

Key Takeaways — TensorFlow & Neural Networks

- **Tensors** are the core data structure (like NumPy arrays).
- **Keras** makes building neural networks simple and readable.

Key Takeaways — TensorFlow & Neural Networks

- **Tensors** are the core data structure (like NumPy arrays).
- **Keras** makes building neural networks simple and readable.
- Neural networks = **layers** + **non-linear activations**.

Key Takeaways — TensorFlow & Neural Networks

- **Tensors** are the core data structure (like NumPy arrays).
- **Keras** makes building neural networks simple and readable.
- Neural networks = **layers** + **non-linear activations**.
- Training loop: forward pass → **loss** → **backprop**.

Key Takeaways — TensorFlow & Neural Networks

- **Tensors** are the core data structure (like NumPy arrays).
- **Keras** makes building neural networks simple and readable.
- Neural networks = **layers** + **non-linear activations**.
- Training loop: forward pass → **loss** → **backprop**.
- Good **preprocessing** is essential for good performance.

Key Takeaways — TensorFlow & Neural Networks

- **Tensors** are the core data structure (like NumPy arrays).
- **Keras** makes building neural networks simple and readable.
- Neural networks = **layers** + **non-linear activations**.
- Training loop: forward pass → **loss** → **backprop**.
- Good **preprocessing** is essential for good performance.

Key Takeaways — TensorFlow & Neural Networks

- **Tensors** are the core data structure (like NumPy arrays).
- **Keras** makes building neural networks simple and readable.
- Neural networks = **layers** + **non-linear activations**.
- Training loop: forward pass → **loss** → **backprop**.
- Good **preprocessing** is essential for good performance.

NumPy + Pandas + TensorFlow = foundation for modern AI